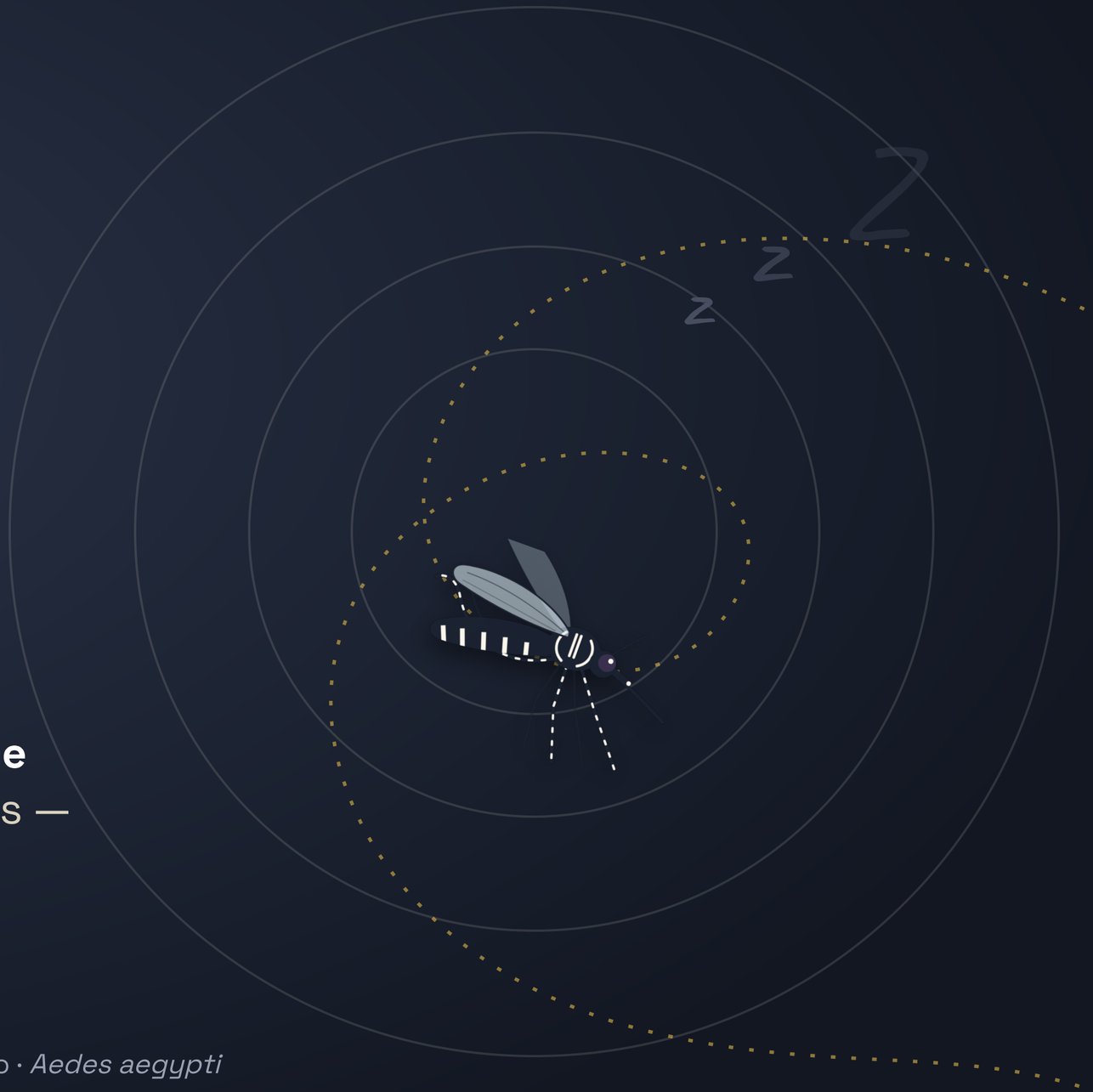
IFSC · IA na Eletrônica · 2026

Zzzz... é *Aedes*?

Como trabalha um desenvolvedor de **Machine Learning**: ferramentas, processo e resultados — ensinando uma máquina a *ouvir* mosquitos.

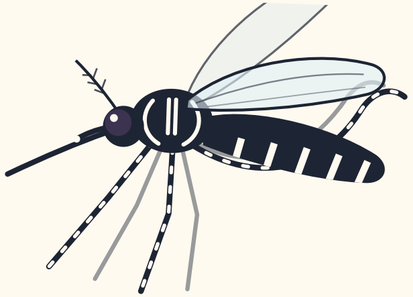


Luiz Eduardo do Nascimento Ciarallo

Projeto `mosquito-wingbeat` · detector ternário: ruído · outro mosquito · *Aedes aegypti*

o problema

Por que ouvir mosquitos?



Um vetor perigoso

O *Aedes aegypti* transmite **dengue, zika e chikungunya**. Saber **onde e quando** ele aparece orienta o combate.



Microfone barato

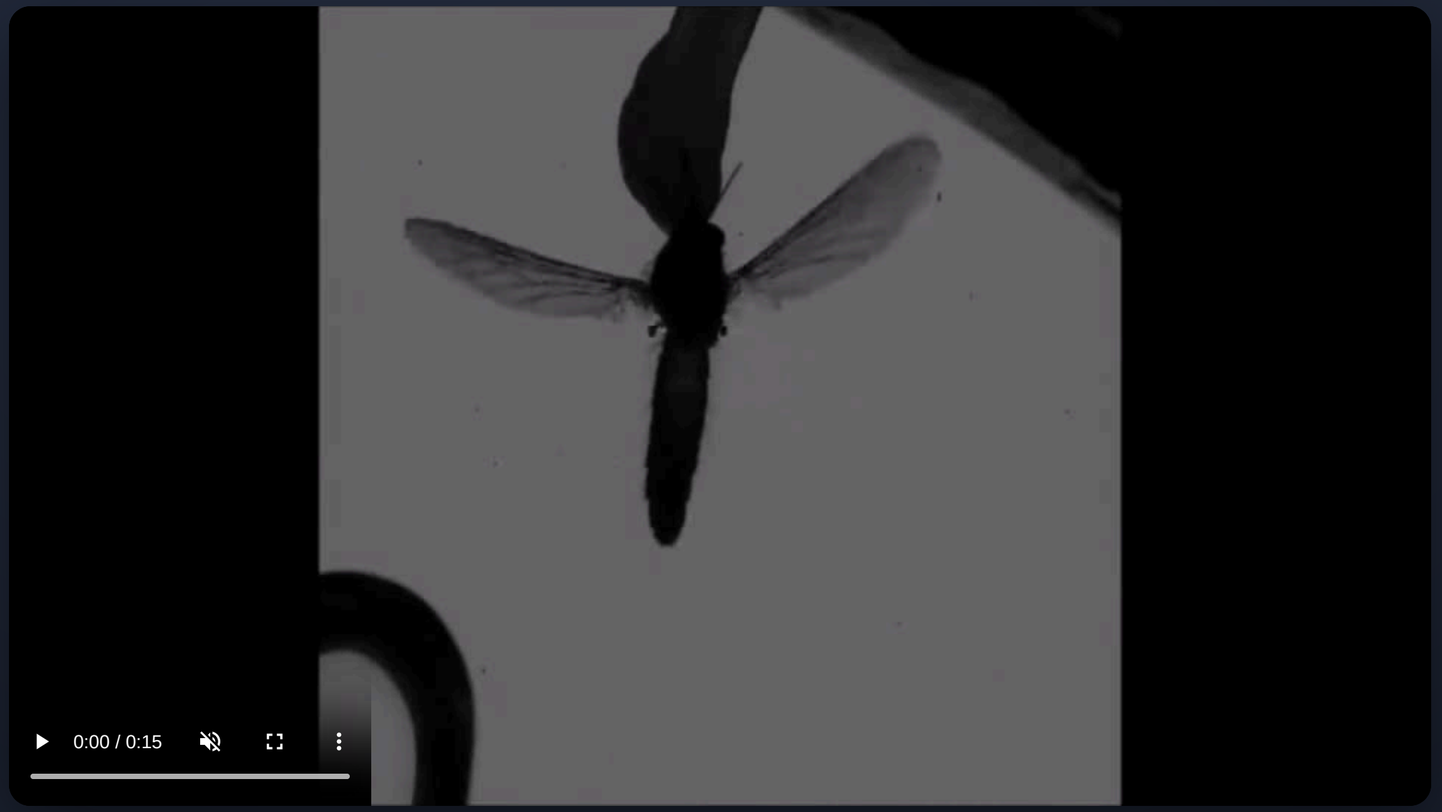
Uma armadilha com microfone pode monitorar **24 h por dia**, sem ninguém olhar lupa. O som vira dado.



~500 batidas de asa por segundo

Cada espécie “canta”

O zumbido é o **batimento de asa**: centenas de Hz, com harmônicos. A ideia é reconhecer a espécie **pelo tom**.



antes da física, olhe

A asa em câmera lenta

Um *Culex tarsalis* preso por cera, filmado a **10.000 quadros por segundo** e mostrado **100× mais devagar**.

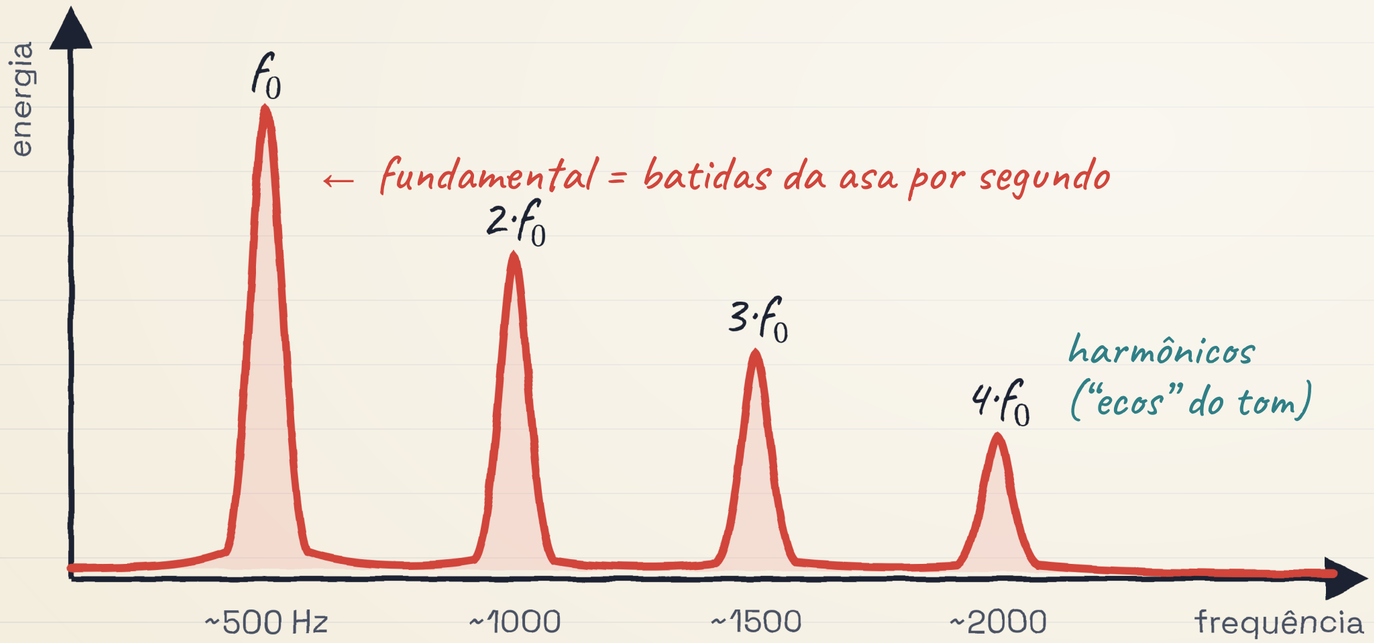
Ao vivo, a asa bate **centenas de vezes por segundo**: rápido demais para o olho, mas é exatamente esse movimento que vira o zumbido.

cada batida = um "empurrão" no ar = som

Vídeo: Mukundarajan, Hol, Castillo, Newby & Prakash, *eLife* 6:e27854 (2017) · CC BY 4.0

a física do zumbido

O canto: um **tom** e seus **ecos**



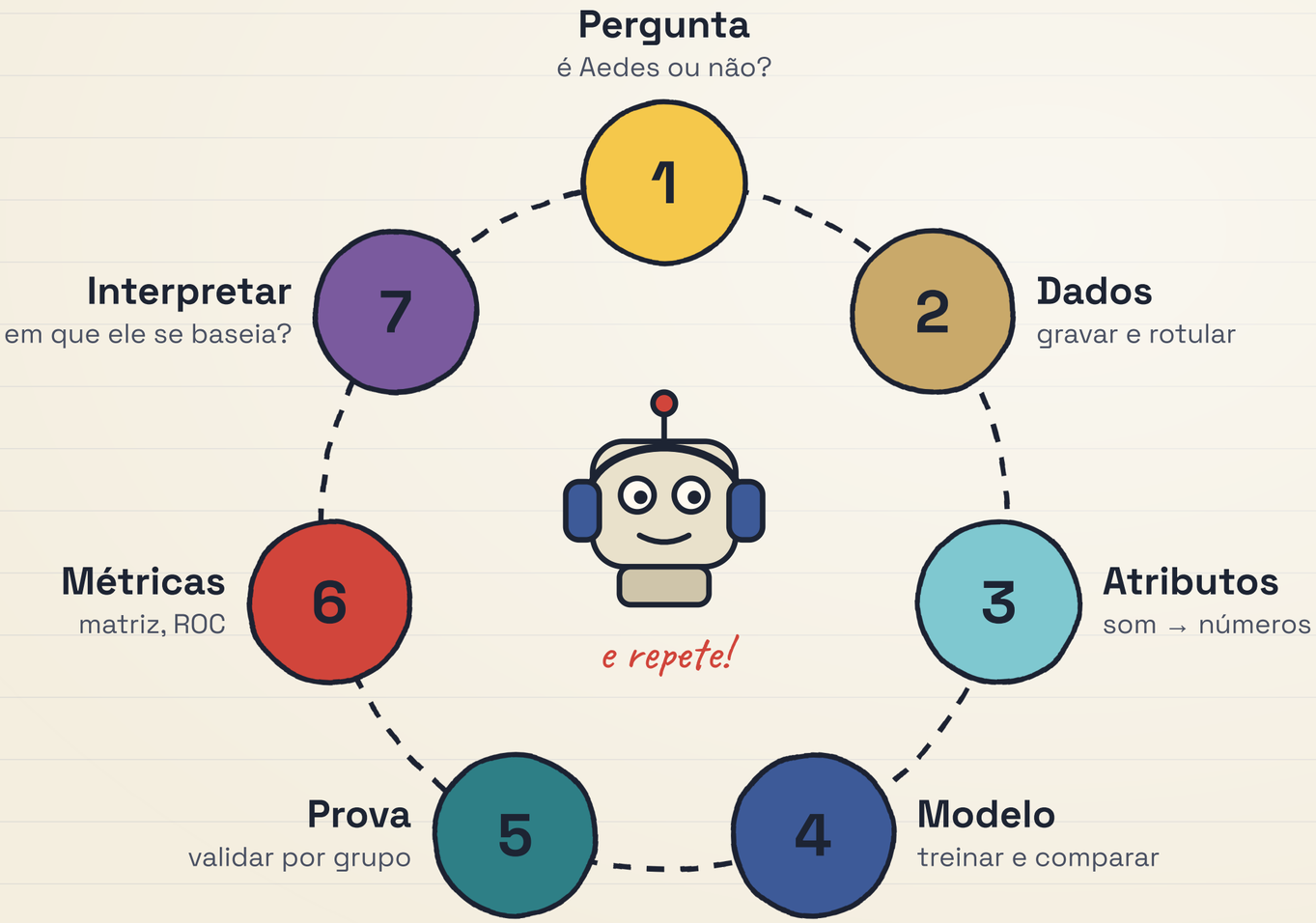
como medimos f_0

```python
# candidatos de 250 a 1000 Hz
cand = np.arange(250, 1001)
# soma harmônica (média dos logs)
harm = np.exp(sum(np.log(amp(k*cand))
          for k in range(1, 4)) / 3)
f0 = cand[np.argmax(harm)]  # o "tom"
```

⚠ Mas os tons **se sobrepõem**: em 80% dos grupos Aedes, f_0 fica entre ~260 e ~580 Hz; nos outros mosquitos, entre ~260 e ~620 Hz. Só o tom não basta.

o trabalho de um dev de ML

Um projeto de ML é um ciclo



Na prática, a maior parte do tempo vai em dados e em provar que funciona — não em escolher o modelo.

Cada volta no ciclo responde a uma pergunta: **"o resultado é real ou o modelo está colando?"** Se a interpretação mostra algo estranho, voltamos aos dados.

é esse ciclo que vamos percorrer com o detector de mosquito

a caixa de ferramentas

As ferramentas do dia a dia

Python
A linguagem de quase todo projeto de ML. Legível e com bibliotecas para tudo.

NumPy · pandas
Tabelas e matrizes de números: cada gravação vira uma linha com 80 colunas.

SciPy · PyTorch
Sinais (espectrograma mel) e as **redes neurais** (MLP e CNN).

scikit-learn
Os modelos (SVM, árvores), a validação cruzada e as métricas, prontos para usar.

Matplotlib
Gráficos: espectros, matriz de confusão, curva ROC. Olhar antes de acreditar.

VS Code · Jupyter
Editor e notebooks: código, texto e gráfico na mesma página.

Git · GitHub
Histórico de cada mudança: dá para voltar atrás e reproduzir um resultado.

Assistentes de IA
Claude Code, Codex e Hermes: escrevem e rodam código comigo. (Volto nisso no fim.)

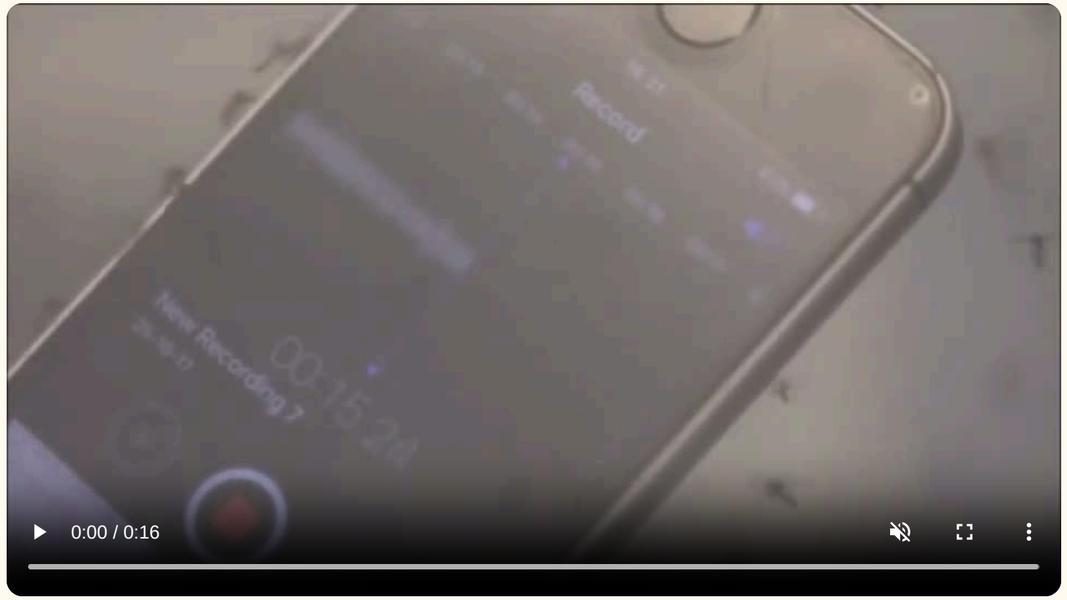
Arduino
E do lado da eletrônica: **Arduino Nano 33 BLE Sense** com microfone embutido, para gravar e testar o detector no mundo real.

de onde vêm os dados

Como se grava um mosquito: com um celular



Um mosquito dentro de uma **garrafa**, com o microfone do celular encostado na boca dela.



Mosquitos **decolando** perto do celular, usando o app de gravador comum.

Foi assim que se gravou o dataset **Abuzz**, *um dos que usamos: celulares comuns, em campo e em laboratório. No nosso protótipo, quem grava é o* **Arduino**.

Vídeo: Mukundarajan, Hol, Castillo, Newby & Prakash, *eLife* 6:e27854 (2017) · CC BY 4.0

os dados

Quem ensina o quê

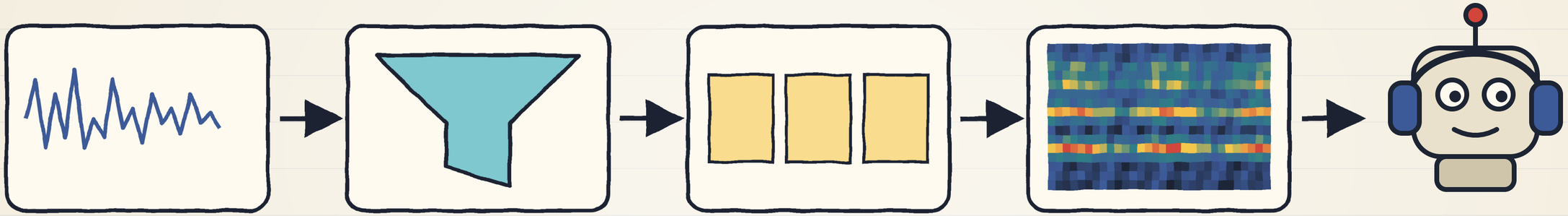
Um modelo de ML aprende **por exemplos rotulados**. A escolha dos exemplos é a primeira decisão de projeto:

1. **Aedes** de dois datasets grandes, **Torres** e **Funchal**: 2.167 exemplos.
2. Os outros datasets ensinam o que **não é Aedes**: 3.050 outros mosquitos e 426 ruídos.
3. Cada exemplo guarda **de onde veio** (fonte e aparelho). Isso vai servir para normalizar e para montar a prova.



como a máquina "ouve"

Do zumbido aos **números**



áudio bruto
.wav, .mp4, celular…

espectrograma mel
energia por faixa

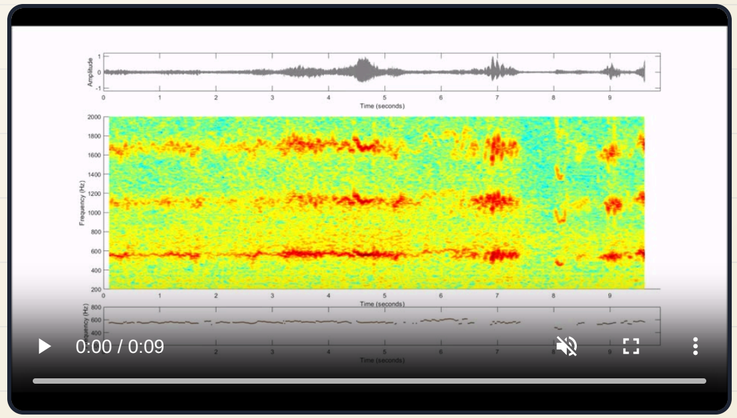
16 bandas até 4 kHz
onde mora o zumbido

5 resumos × 16
= 80 números

modelo
ruído · outro · Aedes

de espectrograma a 80 números

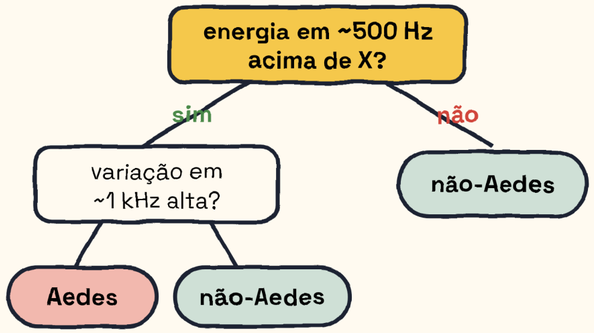
```
z = db[:, :, bandas_ate_4khz]      # mel em dB
z = z - z.max()                    # relativo ao pico
X = np.concatenate([z.mean(1), z.std(1),
     *np.quantile(z, [.1, .5, .9], axis=1)])
# 16 bandas × 5 resumos = 80 atributos
```



Espectrograma de um *Anopheles gambiae* sendo montado ao vivo: as faixas vermelhas são o batimento e seus harmônicos. (Mukundarajan et al., *eLife* 2017, CC BY 4.0)

os modelos de ML

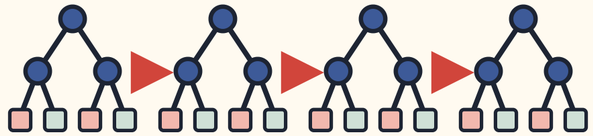
Árvores que fazem perguntas



Árvore de decisão

Uma sequência de perguntas sim/não sobre os 80 números. O treino escolhe **quais perguntas** e **quais limites** separam melhor as classes.

uma depois da outra, corrigindo erros

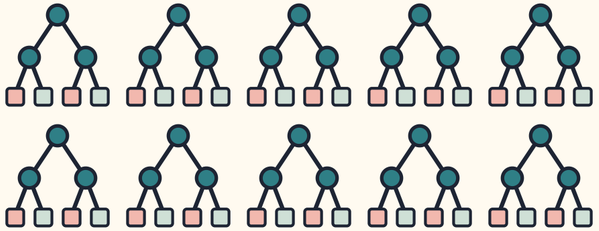


árvore 1 + 2 + 3 + … + 300

HistGB (boosting)

Árvores pequenas **em fila**: cada nova árvore foca nos **erros** das anteriores. A soma de muitas perguntas fracas vira um modelo forte.

todas ao mesmo tempo, depois votam



média de 400 votos

Extra Trees (floresta)

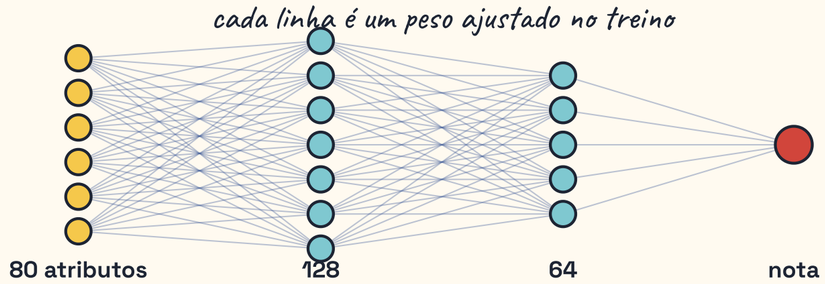
Centenas de árvores **independentes**, com cortes sorteados. Cada uma vota; a média é mais estável que qualquer árvore sozinha.

no scikit-learn

```
hgb = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=300, max_leaf_nodes=15)
et  = ExtraTreesClassifier(n_estimators=400, min_samples_leaf=2)
p_aedes = (hgb.predict_proba(X)[:, 1] + et.predict_proba(X)[:, 1]) / 2   # média dos dois
```

e as redes neurais?

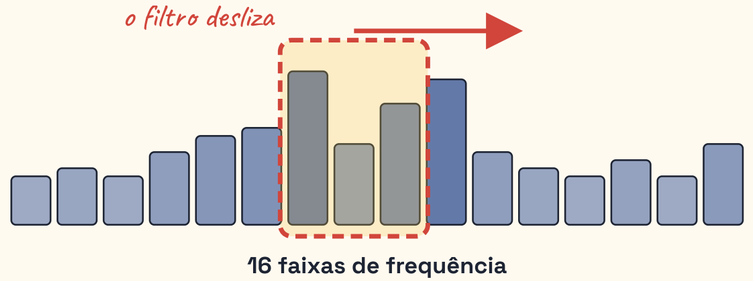
Redes neurais: MLP e CNN



MLP (perceptron multicamadas)

Camadas de “neurônios”: cada um soma os 80 números com **pesos** e passa por uma função (ReLU). O treino ajusta os pesos para errar menos.

```
mlp = nn.Sequential(
  nn.Linear(80, 128), nn.ReLU(), nn.Dropout(0.3),  # camada 1
  nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.3),  # camada 2
  nn.Linear(64, 1))  # nota de Aedes
```



CNN (rede convolucional)

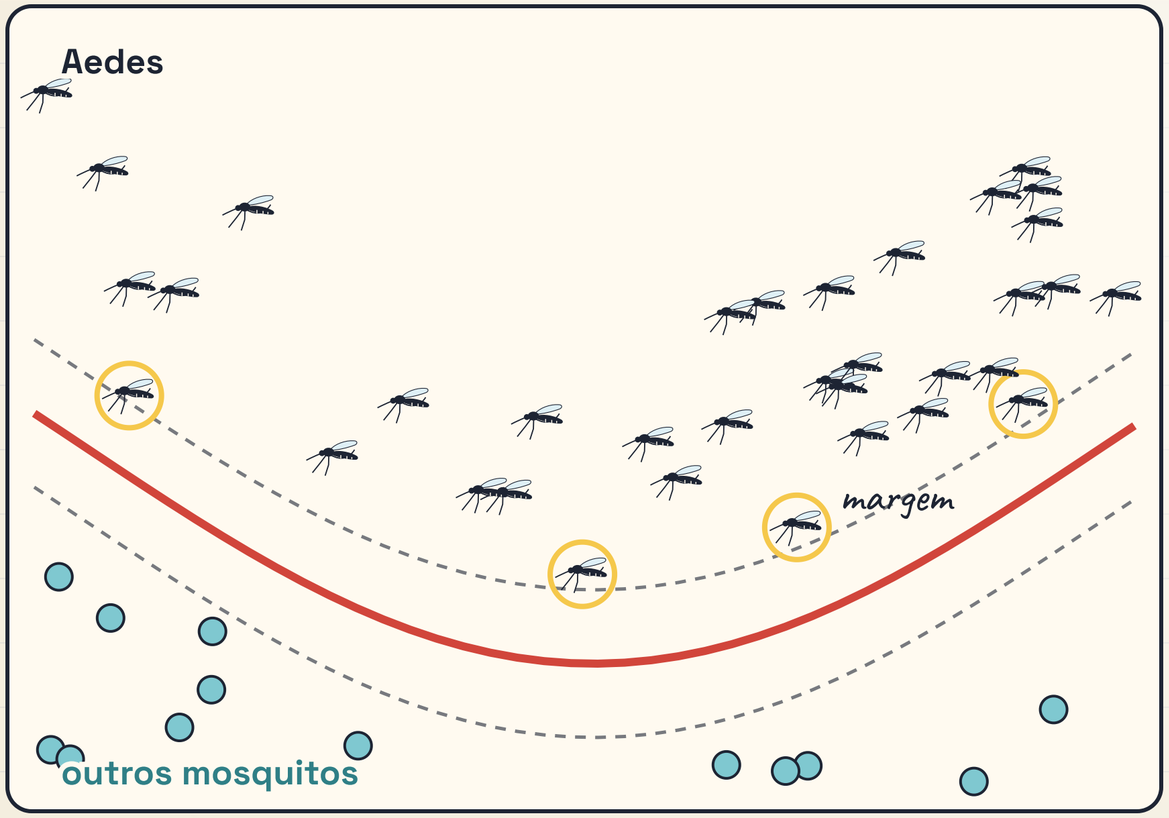
Um **filtro** pequeno desliza pelas 16 faixas de frequência procurando padrões (como “pico + harmônico”), em qualquer posição.

```
cnn = nn.Sequential(
  nn.Conv1d(5, 32, 3, padding=1), nn.ReLU(),  # filtros
  nn.Conv1d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool1d(2),
  nn.Conv1d(32, 64, 3, padding=1), nn.ReLU(), nn.AdaptiveMaxPool1d(1),
  nn.Flatten(), nn.Dropout(0.3), nn.Linear(64, 1))  # nota
```

```
for epoca in range(60):   # treino: mostrar os exemplos várias vezes e corrigir os pesos
    perda = binary_cross_entropy(rede(X_lote), y_lote); perda.backward(); otimizador.step()
```

o modelo final

SVM: a fronteira com mais folga



○ *amarelo = vetores de suporte (os que definem a fronteira)*

Como funciona

A SVM (*máquina de vetores de suporte*) procura a linha que separa as classes deixando a **maior margem** possível. Só os pontos na beira da margem — os **vetores de suporte** — definem a fronteira.

O truque do kernel RBF

Com 80 números, uma reta não basta. O kernel RBF mede a **semelhança** entre gravações e permite fronteiras **curvas**.

```
from sklearn.svm import SVC
svm = SVC(kernel='rbf', C=1.0, probability=True)
svm.fit(X_treino, y_treino, sample_weight=pesos)
p_aedes = svm.predict_proba(X_teste)[:, 1]
```

*A SVM mede **distâncias**, então precisa de atributos na mesma escala — exatamente o que a normalização por fonte entrega.*

a regra de ouro

Não deixar o modelo colar na prova

✗ dobras por arquivo



o mesmo celular aparece nos dois lados

✓ dobras por pote (cluster)



a prova é de um aparelho nunca visto

o "fiscal" da prova

```
sg = StratifiedGroupKFold(n_splits=5)
for tr, te in sg.split(X, y, groups=cluster):
    assert not (set(cluster[tr])
                & set(cluster[te])), \
        "cluster leakage"
```

Se o mesmo aparelho aparece no treino e na prova, o modelo pode **decorar o aparelho** e a nota fica inflada.

Grupo = gravação, indivíduo ou aparelho. **5 dobras**, repetidas **3 vezes** com sorteios diferentes.

como foi feito

Três ajustes no treino



cada aparelho tem seu "sotaque"

1 Normalizar por fonte

Cada fonte é centralizada nela mesma: some o "sotaque" do microfone, fica o mosquito.

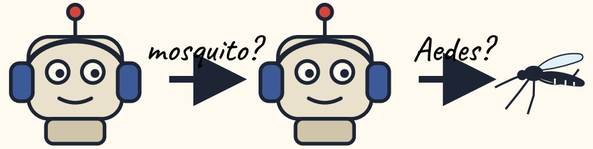
```
for fonte in fontes:
    m = origem == fonte
    X[m] = (X[m] - X[m].mean(0)) \
           / X[m].std(0)
```



2 Pesos por fonte

Uma fonte com 1.800 exemplos não pode "gritar" mais alto que uma com 20. Cada fonte pesa igual dentro da classe.

```
for fonte in fontes_da_classe:
    m = (y == c) & (origem == fonte)
    w[m] = 1 / m.sum() / n_fontes
modelo.fit(X, y, sample_weight=w)
```



3 Dois estágios

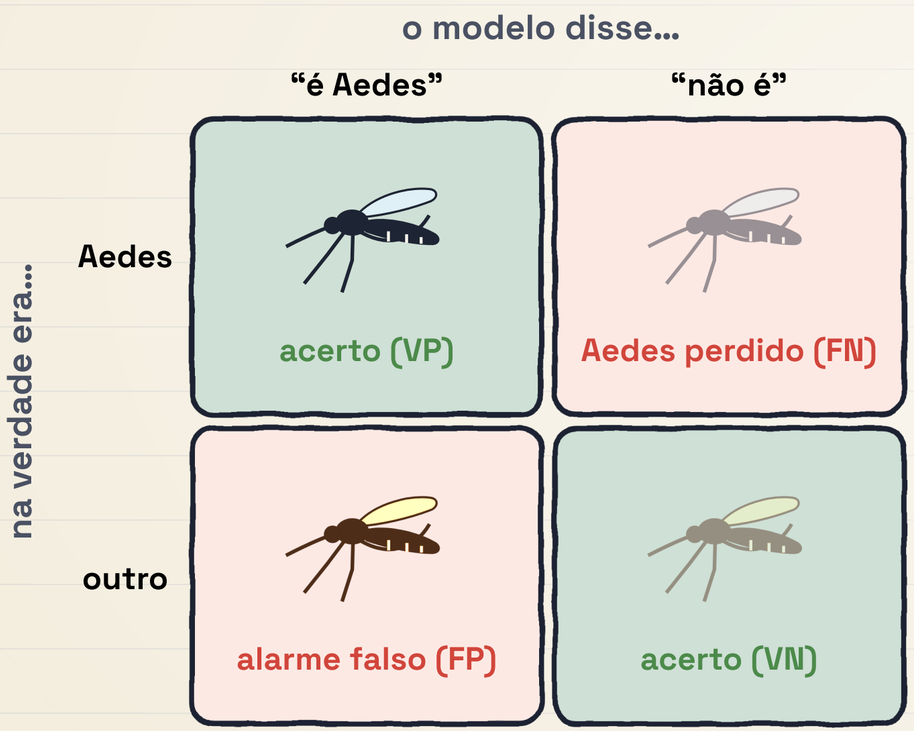
Primeiro: **tem mosquito?** Depois, só entre mosquitos: **é Aedes?** Um modelo para cada pergunta.

```
p_m = svm_mosquito.predict_proba(X)[:, 1]
p_a = svm_aedes.predict_proba(X)[:, 1]

p = [1 - p_m,           # ruído
     p_m * (1 - p_a),   # outro
     p_m * p_a]         # Aedes
```

como medir

Acurácia não basta: três perguntas



a “matriz de confusão”

Precisão = VP / (VP + FP)

Quando o modelo apita “Aedes”, com que frequência acerta?

Detecção (recall) = VP / (VP + FN)

De todos os Aedes que existiam, quantos ele encontrou?

AUROC

Sorteie um Aedes e um não-Aedes: qual a chance de o Aedes ganhar nota maior? **0,5 = moeda**, 1 = perfeito. Não depende do limiar.

```
from sklearn.metrics import precision_score, recall_score, roc_auc_score
precision_score(y, y_pred); recall_score(y, y_pred)
roc_auc_score(y, p_aedes)   # usa a probabilidade, não o rótulo
```

com classes desbalanceadas, “acertar 90%” pode significar só chutar a classe mais comum

os métodos, em código

Cada método em poucas linhas

1 · boosting

HistGradientBoosting

Árvores pequenas em fila; cada uma corrige os erros da anterior.

```
from sklearn.ensemble import (
    HistGradientBoostingClassifier)
hgb = HistGradientBoostingClassifier(
    learning_rate=0.05, max_iter=300)
hgb.fit(X_treino, y_treino)
p = hgb.predict_proba(X_teste)[:, 1]
```

2 · margem

SVM (modelo final)

Procura a fronteira que separa as classes com a **maior margem**; o kernel RBF deixa a fronteira curva.

```
from sklearn.svm import SVC
svm = SVC(kernel='rbf', C=1.0, probability=True)
svm.fit(X_treino, y_treino, sample_weight=w)
p = svm.predict_proba(X_teste)[:, 1]
```

3 · cascata

Dois estágios

Primeiro “tem mosquito?”, depois, só entre mosquitos, “é Aedes?”.

```
p_mosq  = svm_mosquito.predict_proba(X)[:, 1]
p_aedes = svm_aedes.predict_proba(X)[:, 1]
p_final = p_mosq * p_aedes  # nota de Aedes
```

4 · validação

Validação cruzada por grupo

5 rodadas; em cada uma, aparelhos inteiros ficam fora do treino.

```
from sklearn.model_selection import (
    StratifiedGroupKFold)
cv = StratifiedGroupKFold(n_splits=5)
for tr, te in cv.split(X, y, aparelho):
    modelo.fit(X[tr], y[tr])
    P[te] = modelo.predict_proba(X[te])
```

5 · métricas

Matriz de confusão e AUROC

Comparam o que o modelo disse com o que era de verdade.

```
from sklearn.metrics import (
    confusion_matrix, roc_auc_score)
confusion_matrix(y, y_previsto)  # 3×3
roc_auc_score(y == 2, p_aedes)  # 0,83
```

6 · interpretação

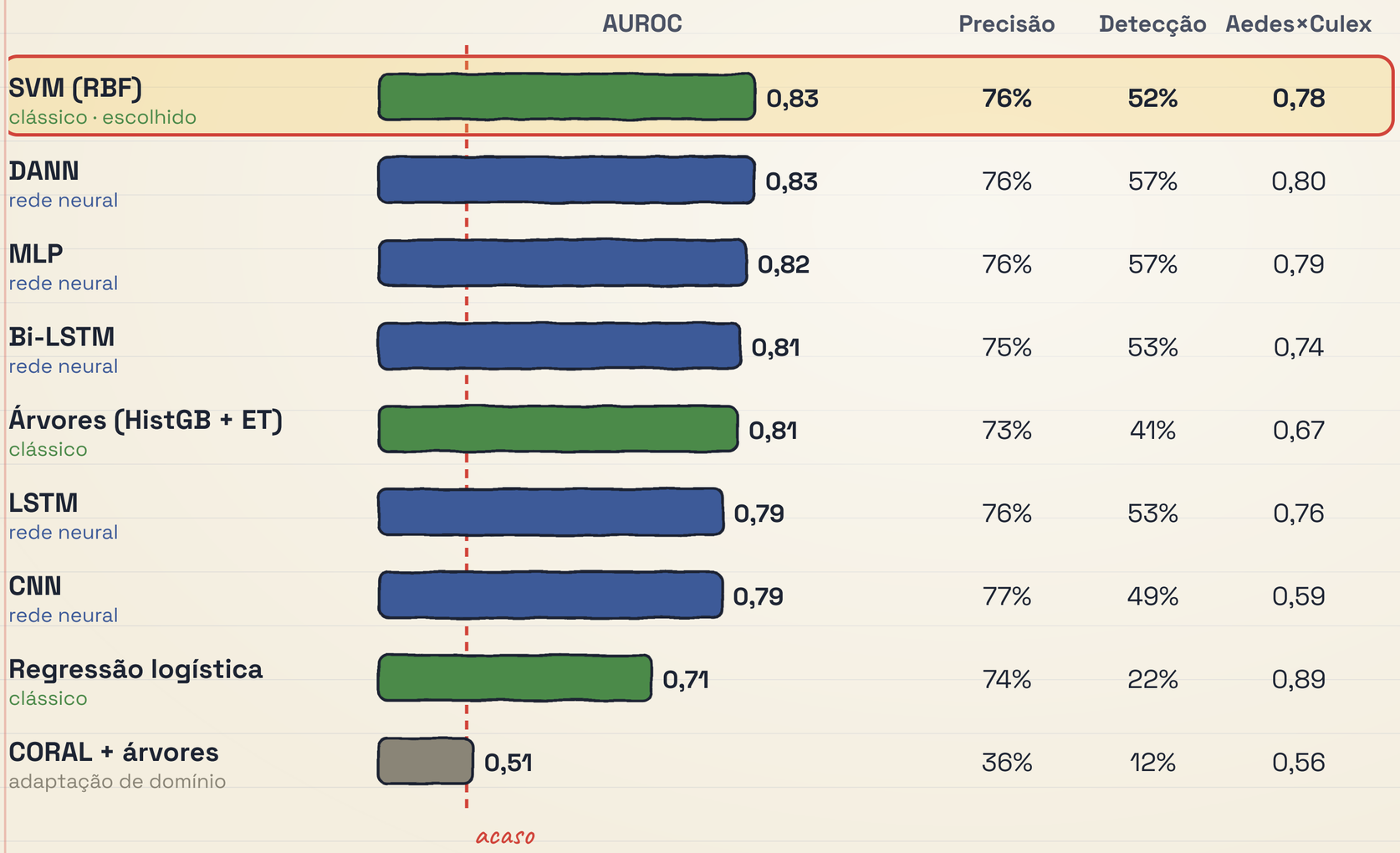
Importância por permutação

Embaralha uma faixa de frequência e mede quanto o modelo piora.

```
Z = X_teste.copy()
Z[:, faixa] = rng.permutation(Z[:, faixa])
p = modelo.predict_proba(Z)[:, 1]
queda = auc_base - roc_auc_score(y, p)
```

qual modelo escolher?

9 modelos, a mesma prova



	Precisão	Detecção	Aedes×Culex
SVM (RBF) clássico · escolhido	**76%**	**52%**	**0,78**
DANN rede neural	76%	57%	0,80
MLP rede neural	76%	57%	0,79
Bi-LSTM rede neural	75%	53%	0,74
Árvores (HistGB + ET) clássico	73%	41%	0,67
LSTM rede neural	76%	53%	0,76
CNN rede neural	77%	49%	0,59
Regressão logística clássico	74%	22%	0,89
CORAL + árvores adaptação de domínio	36%	12%	0,56

Escolhemos a SVM

Lidera o AUROC (0,83) com a **menor variação** entre repetições, treina em segundos na CPU e é simples de explicar. Mas **não vence em tudo**: DANN e MLP empatam no AUROC e detectam um pouco mais.

Redes neurais (DANN, MLP, Bi-LSTM) ficaram muito perto; a **CNN** e a **LSTM** ficaram atrás, e o **CORAL** atrapalhou (AUROC 0,51, igual ao acaso).

O modelo mais novo não é sempre o melhor: teste vários com a mesma prova e escolha pelos números.

mesmos dados, mesmos ajustes (normalização, pesos, dois estágios), validação por grupo · média de 3 repetições

o resultado, em uma tabela

A matriz de confusão do nosso modelo

era de verdade… \ o modelo disse…	ruído	outro mosquito	Aedes
ruído	**225** 53% da linha	**184** 43% da linha	**17** 4% da linha
outro mosquito	**163** 5% da linha	**2.554** 84% da linha	**333** 11% da linha
Aedes	**9** 0% da linha	**1.027** 47% da linha	**1.131** 52% da linha

Diagonal = acertos. 1.131 Aedes, 2.554 outros mosquitos e 225 ruídos.

O erro mais comum: 1.027 Aedes chamados de "outro mosquito". O modelo é **cauteloso**.

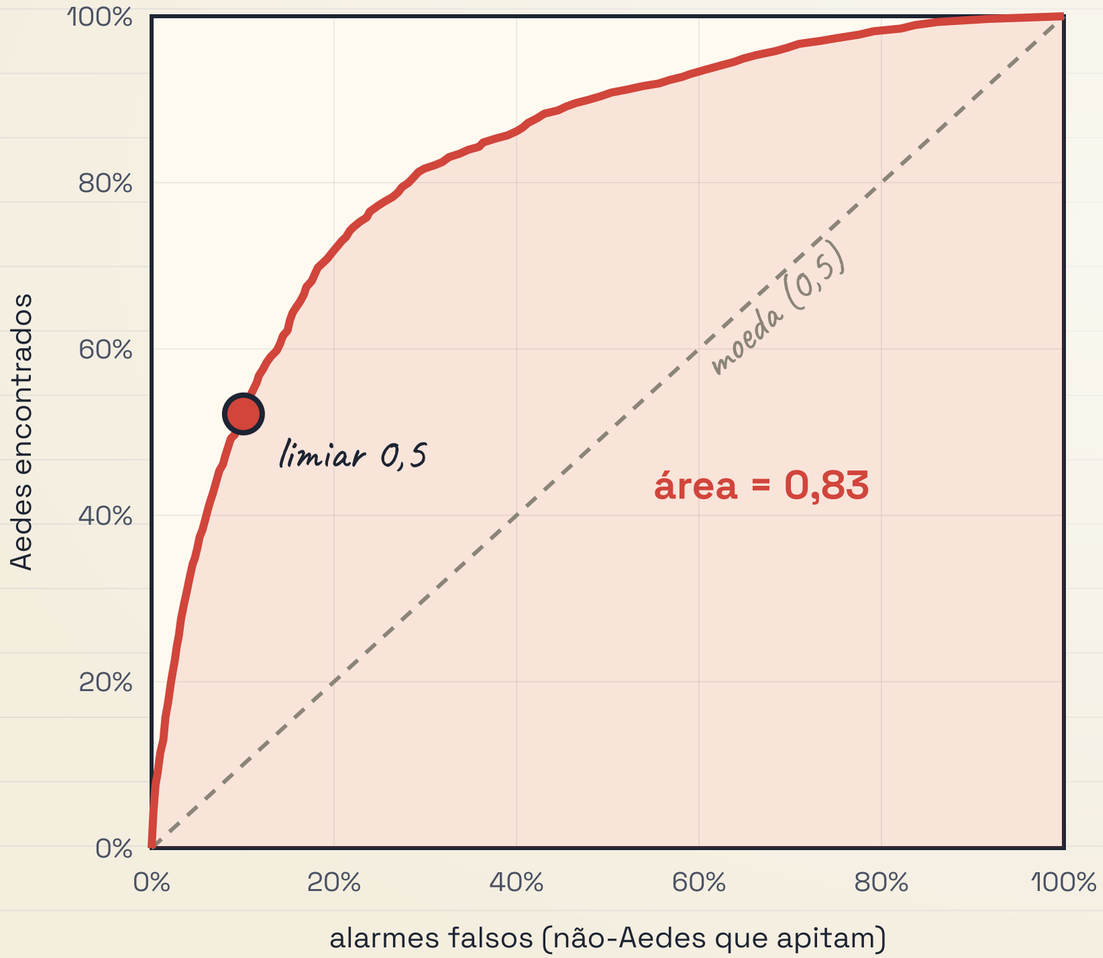
Alarmes falsos: 333 de 3.050 outros mosquitos (11%) e 17 de 426 ruídos (4%).

Precisão = 1.131 / 1.481 = 76%
Detecção = 1.131 / 2.167 = 52%

5.643 gravações, cada uma prevista por um modelo que nunca a viu · limiar 0,5 · modelo: SVM

curvas que ajudam a decidir

A curva ROC



Como ler

Cada ponto é um **limiar** diferente. Para cima = mais Aedes encontrados; para a direita = mais alarmes falsos. A diagonal é o chute (moeda).

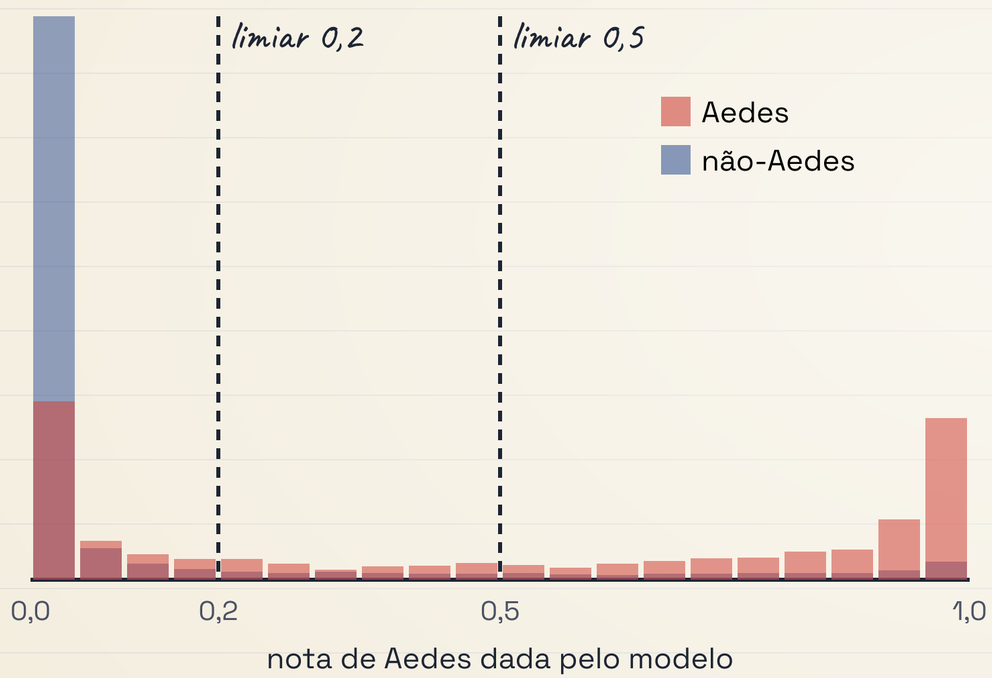
AUROC = 0,83

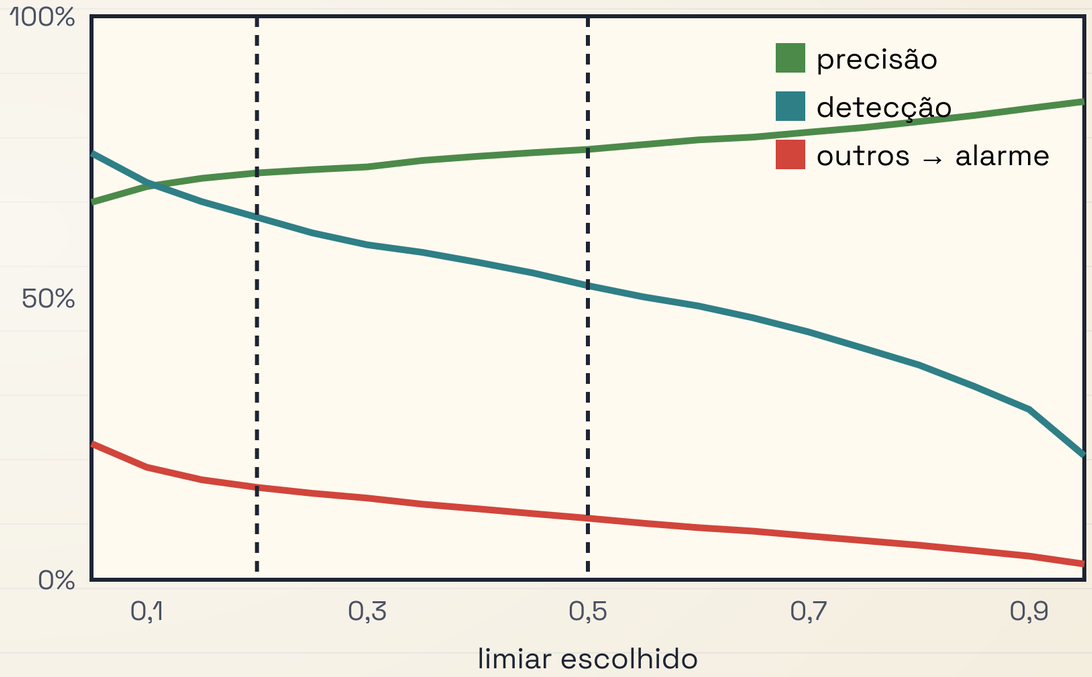
A área sob a curva. Escolhendo um Aedes e um não-Aedes ao acaso, em **83%** das vezes o Aedes recebe nota maior.

O ponto vermelho é o limiar 0,5 que usamos: 52% dos Aedes com 11% de alarmes falsos nos outros mosquitos.

uma decisão de projeto

Onde cortar a nota?





O modelo não diz "sim/não": dá uma nota de 0 a 1. Quem escolhe o corte somos nós.

Limiar 0,2 (vigilância, não perder Aedes): 64% detectados, 72% de precisão.

Limiar 0,5 (poucos alarmes): 52% detectados, 76% de precisão, 11% dos outros mosquitos apitam.

esquerda: notas que o modelo dá · direita: as três métricas para cada limiar

resultado

Quando apita, costuma ser Aedes

0,83	76%	11%	4%
AUROC para Aedes	dos alertas de Aedes estão certos	dos outros mosquitos viram alarme	dos ruídos viram alarme

De cada 100 alertas de “Aedes”...



76 são Aedes de verdade 24 são outro mosquito ou ruído

e 11% dos outros mosquitos (4% dos ruídos) disparam alarme

*Quando o modelo diz “é Aedes”, ele acerta **cerca de 3 em cada 4** vezes.*

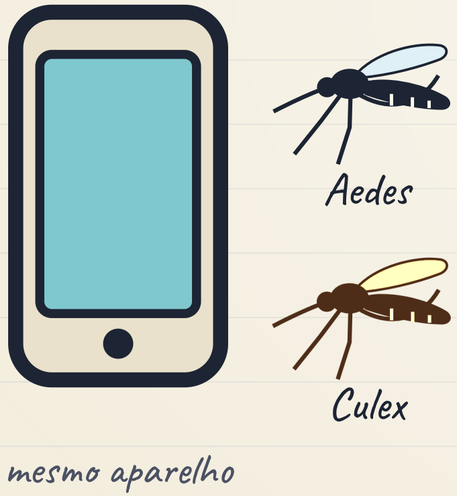
O modelo é **cauteloso**: prefere ficar quieto a dar alarme falso. Detecta **52%** dos Aedes, mas quando apita, costuma estar certo.

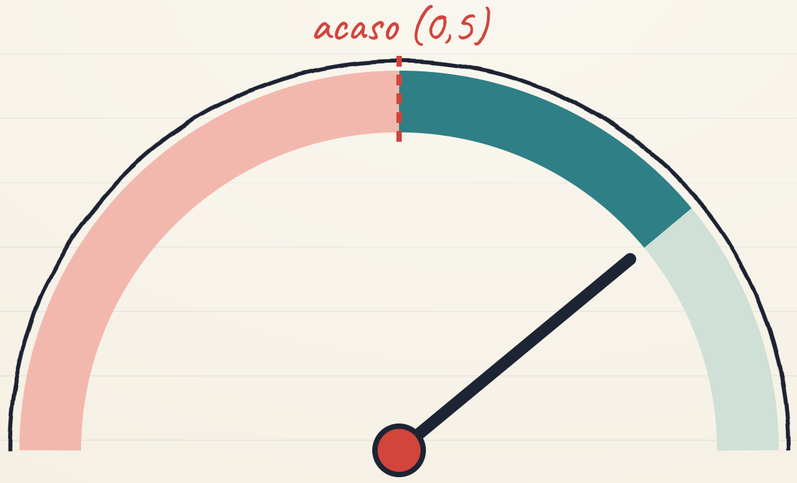
média de 3 repetições · validação cruzada por grupo · modelo: SVM

a prova mais justa

Aedes × *Culex* no mesmo aparelho

Aedes e *Culex* gravados nos **mesmos aparelhos.** Deixamos um aparelho inteiro fora do treino e perguntamos: o modelo separa as duas espécies?





0,78

AUROC Aedes × Culex · aparelho nunca visto

Por que essa prova vale mais

O aparelho é o mesmo para as duas espécies, então **o "sotaque" do microfone não ajuda.** Para acertar, o modelo precisa ouvir o mosquito.

Ainda falta

O modelo só viu Aedes de **dois datasets.** Aedes gravados em **outros aparelhos** ainda não são reconhecidos (AUROC 0,51, igual ao acaso): o próximo passo é coletar mais fontes.

interpretabilidade

O que o modelo está **ouvindo**?

As métricas dizem **quanto** o modelo acerta.
A interpretabilidade pergunta **em que ele se baseia**.

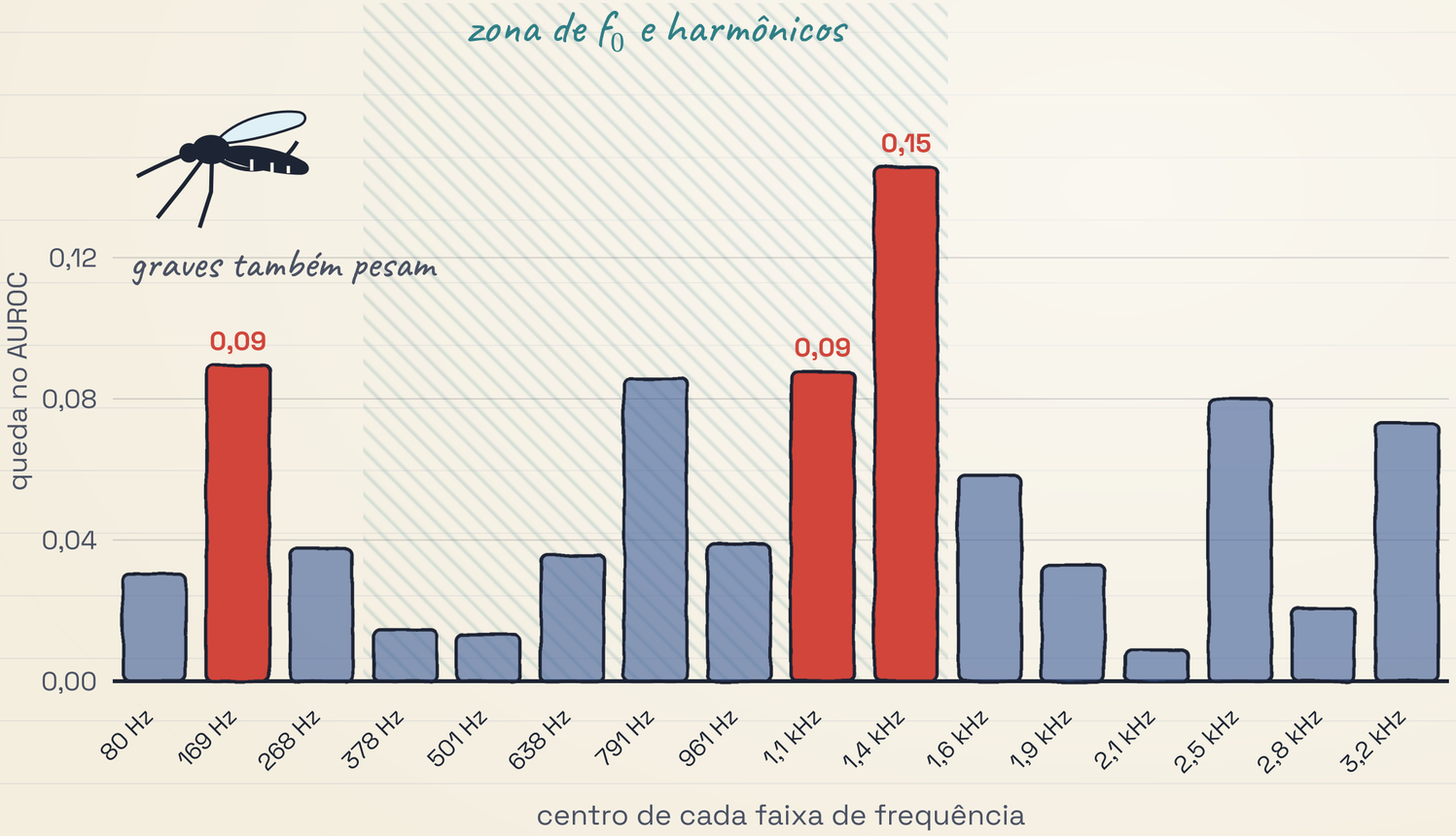
🎧 quais frequências ele usa?

🔍 ele ainda “ouve” o aparelho?



importância por permutação

Ele usa o espectro todo



embaralhar uma faixa

```
for banda in range(16):
    Z = X_teste.copy()
    c = colunas(banda)  # 5 resumos
    Z[:, c] = Z[sorteio][:, c]
    queda[banda] = auroc_base \
        - roc_auc_score(y, modelo(Z))
```

Se embaralhar uma faixa **derruba** o AUROC, o modelo depende dela. A maior queda está em **~1,4 kHz** (perto de $3 \cdot f_0$), mas graves (170 Hz) e agudos (2,5 e 3,2 kHz) também pesam: parte do que o modelo usa pode ser fundo ou aparelho.

E o aparelho? Antes da normalização, um classificador linear adivinha de qual fonte veio um mosquito em **50%** das vezes; depois, em **14%** (acaso: 20%).

modelo final (SVM) · queda de AUROC no teste ao embaralhar cada faixa (16 bandas mel)

IA generativa ajudando a fazer IA

Como usei assistentes de IA no código

Claude Code
Anthropic

Codex
OpenAI

Hermes
Nous Research

DeepSeek
e outros modelos

1 Eu descrevo
o objetivo, em português

2 A IA escreve
e roda o código

3 Eu confiro
números, gráficos, testes

4 Ajustamos
e repetimos

Onde ajudou muito

Código repetitivo, gráficos, ler arquivos de vários datasets, testar várias ideias rápido, revisar e documentar.

Onde precisei conferir

Se treino e prova estavam mesmo separados, se os arquivos foram lidos certo e se cada número fazia sentido.

```
assert not set(grupo[tr]) & set(grupo[te])
```

A regra

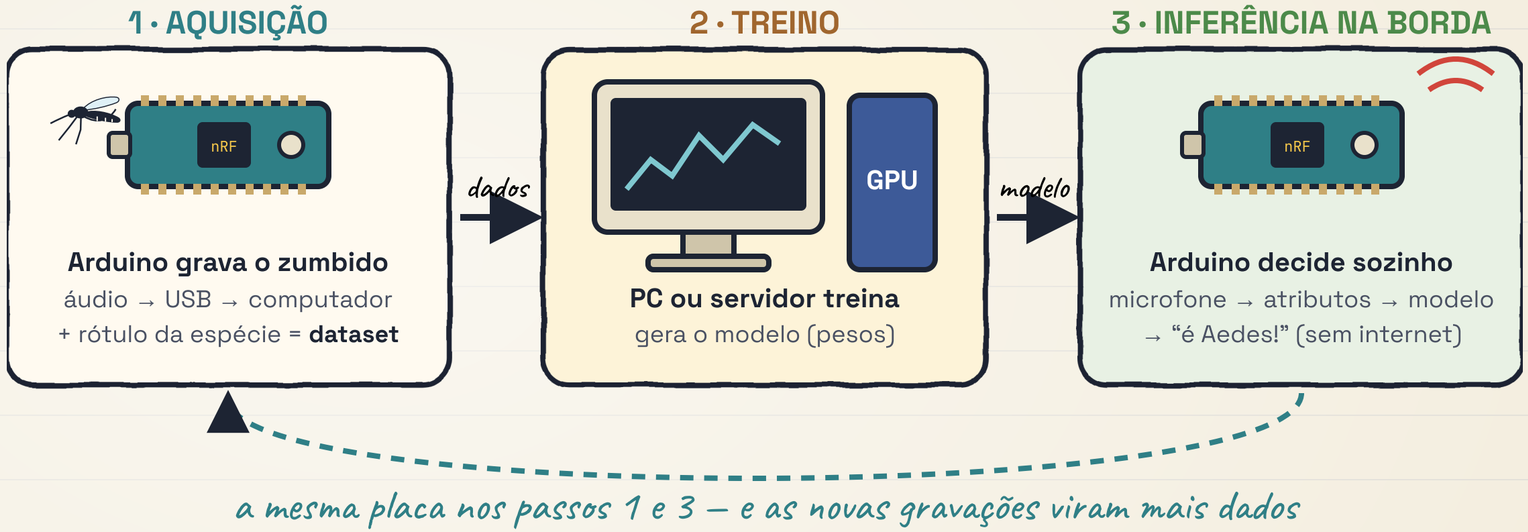
A IA acelera, mas **a responsabilidade pelo resultado é sua.** Peça para ela explicar o código e rode testes que poderiam dar errado.

IA na eletrônica

A mesma placa, dois papéis



o protótipo: a placa num pote-armadilha



Arduino Nano 33 BLE Sense

⚙️ **nRF52840**: ARM **Cortex-M4F** a **64 MHz**, com FPU (contas com vírgula em hardware)
🧠 **256 KB** de RAM · **1 MB** de flash para o programa e o modelo
🎤 microfone **MEMS digital** (PDM) já na placa
📡 **Bluetooth LE** · 🔋 consumo baixo, roda com bateria

Treinar é pesado → PC / servidor

Passa por **milhares de exemplos, dezenas de vezes**, calculando erros e corrigindo pesos. Precisa de GB de memória e, de preferência, GPU. Leva minutos ou horas.

Inferir é leve → microcontrolador

Só **uma gravação por vez** e só a "ida" do modelo (somas e multiplicações). Ex.: a MLP do slide 9 tem 18.689 pesos ≈ **73 KB** e cabe no 1 MB de flash.

próximos passos

Receita para um teste de verdade

- [ ] **Controles pareados:** Aedes *e* outras espécies gravados no **mesmo aparelho, setup e dia** — em vários aparelhos e locais.
- [ ] **Ruído in situ** dos ambientes reais, no lugar de ESC-50/Pete.
- [ ] **Mais fontes de Aedes:** o modelo só viu Aedes de dois datasets e ainda não reconhece Aedes de aparelhos novos.
- [ ] **Um teste cego único**, numa campanha nova, com o protocolo congelado **antes** da coleta.



de onde veio tudo isso

Referências

Dados e trabalhos sobre mosquitos

1. Mukundarajan H., Hol F.J.H., Castillo E.A., Newby C., Prakash M. *Using mobile phones as acoustic sensors for high-throughput mosquito surveillance*. **eLife** 6:e27854, 2017. doi:10.7554/eLife.27854 — dataset Abuzz e vídeos (CC BY 4.0).
2. Kiskin I. et al. *HumBugDB: A Large-scale Acoustic Mosquito Dataset*. NeurIPS Datasets and Benchmarks, 2021. arXiv:2110.07607.
3. Sinka M.E. et al. *HumBug – An Acoustic Mosquito Monitoring Tool for use on budget smartphones*. **Methods Ecol. Evol.** 12, 2021. doi:10.1111/2041-210X.13663.
4. Torres I., Nakano M., Cime-Castillo J. et al. *Dengue-Infected Mosquito Detection with Uncertainty Evaluation based on Monte Carlo Dropout*. **IEEE Latin America Trans.** 23(2):135–143, 2025.
5. Vasconcelos D., Nunes N.J., Gomes J. *An annotated dataset of bioacoustic sensing and features of mosquitoes*. **Scientific Data** 7:382, 2020 — dataset Funchal.
6. Supratak A. et al. *MosquitoSong+: A noise-robust deep learning model for mosquito classification from wingbeat sounds*. **PLOS ONE** 19, 2024. doi:10.1371/journal.pone.0310121.
7. Paim K.O., Rohweder R., Recamonde-Mendoza M., Mansilha R.B., Cordeiro W. *Acoustic Identification of Ae. aegypti Mosquitoes using Smartphone Apps and Residual CNNs*. arXiv:2306.10091, 2023 — dados UFRGS.
8. Piczak K.J. *ESC: Dataset for Environmental Sound Classification*. ACM Multimedia, 2015 — ruídos ESC-50.

Métodos, ferramentas e hardware

1. Cortes C., Vapnik V. *Support-vector networks*. **Machine Learning** 20:273–297, 1995 — SVM.
2. Friedman J.H. *Greedy function approximation: a gradient boosting machine*. **Ann. Statist.** 29(5), 2001; Ke G. et al. *LightGBM*. NeurIPS, 2017 — HistGB.
3. Geurts P., Ernst D., Wehenkel L. *Extremely randomized trees*. **Machine Learning** 63:3–42, 2006.
4. Hochreiter S., Schmidhuber J. *Long short-term memory*. **Neural Computation** 9(8), 1997 — LSTM.
5. Ganin Y. et al. *Domain-adversarial training of neural networks*. **JMLR** 17, 2016 — DANN.
6. Sun B., Feng J., Saenko K. *Return of frustratingly easy domain adaptation*. AAAI, 2016 — CORAL.
7. Breiman L. *Random forests*. **Machine Learning** 45:5–32, 2001 — importância por permutação.
8. Pedregosa F. et al. *Scikit-learn: Machine learning in Python*. **JMLR** 12, 2011; Paszke A. et al. *PyTorch*. NeurIPS, 2019.
9. Arduino. *Nano 33 BLE Sense Rev2 — documentação*. docs.arduino.cc; Nordic Semiconductor. *nRF52840 Product Specification*.

conclusão

O que aprendemos ouvindo mosquitos

1

Dados primeiro, depois o modelo: os ajustes de dados cortaram os alarmes falsos de 28% para 11%; entre **9 modelos** na mesma prova, vários empatam no topo e a **SVM** ganha pela estabilidade.

2

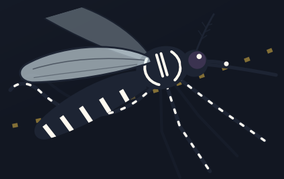
Prova honesta: validar por grupo e aparelho. Resultado: **76% de precisão**, AUROC **0,83**; Aedes × Culex no mesmo aparelho: **0,78**.

3

Interpretabilidade: o maior peso está em **~1,4 kHz**, mas graves e agudos também contam. Depois de normalizar, a fonte deixa de ser adivinhável (**14%**, acaso 20%).

Obrigado!

perguntas? bzzz...



Luiz Eduardo do Nascimento Ciarallo · `mosquito-wingbeat`